A BM MANUAL

RIFF: writing music for bm

Write rhythms and melodies in a few lines, hear them at once, and use them in your game.

Contents

What RIFF is 4

What it does 4

Why it matters 4

Good to know 4

Your first song in 5 minutes 5

The notation in full 7

The mini-notation 7

Words, notes and numbers 7

Building patterns 8

Changing patterns 8

Scales and signals 9

Instruments and effects 10

Choosing the instrument 10

The sound of each note 10

The kit 11

Patterns, songs and structure 12

Several parts together 12

Verse and chorus 12

Repeating and changing 12

Tempo 12

Turning a pattern into a song of the bank 12

Using a RIFF song in a game 13

The three calls every game needs 13

What a slow frame does 13

The functions of the game 13

Live code from the game 14

Voices, sound effects and bank songs 14

Examples 15

A beat, a bass and keys 15
A pad, an arpeggio and drums 15
A melody on a scale 15
The kit and the effects 15
One-liners for the F7 line 15
Stopping and starting from code 16

Common mistakes 17

Open points 18

README

What this is. The manual of RIFF, bm's language for writing music in a few lines of Lua. The content comes from `docs/RIFF.md`, `docs/api/riff.md` and `tests/riff/cart.lua`.

How to use it. Do the first song (chapter 2) step by step, then keep the notation chapters at hand. Anything in a bordered box with a shadow is meant to be copied as it is.

What is missing. Every gap is marked with a highlighted TO FILL tag and listed in the last chapter; nothing is invented.

What RIFF is

RIFF is a way to write music as text. A rhythm or a melody is a short string; functions that chain together change it. The style comes from TidalCycles and Strudel.

What it does

- You write `s "kick*4, ~ snare, hat*8"` and you get a drum beat.
- You change it with methods: `:fast(2)`, `:gain(.9)`, `:room(.4)`.
- It plays with bm's own synthesizer and instruments, and with the sounds of the cartridge's bank, by name.

Why it matters

A song for a game is a few lines, not a file of notes. You can write it and hear it live in **bm Code**, turn it into a song of the bank in **bm Sound**, or play it from your game while it runs.

Good to know

- RIFF is a library of the kernel (`src/script/riff.lua`, like bmlib). You load it with `require "riff"`. It runs in `.bm` files, on the Pi and on the RGB30, and in bmhost on the PC.
- Time is counted in **cycles**. A cycle is a bar of four beats: 2 seconds at 120 BPM (`setcps(0.5)`, the initial value).
- A pattern says which events fall in an interval of cycles. Every frame, `R.update()` queues the notes of the next moments (about 0.2 s ahead) with `play_at()`. The sound interrupt starts them on time, whatever the frame rate of the game.
- Notes wait in a queue of up to 160. RIFF never takes the voices of a bank song or of a sound effect.

Your first song in 5 minutes

You will write a beat, hear it, add a bass line, and hear both. You need bm Sound open on a bank (the F7 line plays with the sounds of the open bank and the instruments).

> **TRY IT**
> Go through the steps in order. After each step, look at what the screen should show.

1. Open **bm Sound**. **[TO FILL: how to start bm Sound from the console menu]**
2. Press **F7** (or choose *Riff...* in the menu). A line opens where you write a pattern, with a first example already in it. You can move through eight examples with the up and down keys.
3. Erase the example and type this line:

```
s "kick*4, ~ snare, hat*8"
```

This is a bar of four beats: four kicks, a snare on the second half, eight hats.

4. Press **Enter**. You hear the beat and the status line says `playing`. The words of the notes that sound light up.
5. Make it softer and add a room. Change the line to:

```
s "kick*4, ~ snare, hat*8" :gain(.9) :room(.2)
```

Press **Enter** again: the beat plays with the change.

6. Try a melody instead. Type this and press **Enter**:

```
note "c4 e4 g4 c5" :s "epiano" :room(.4)
```

You hear four notes of a C major chord on the electric piano, one every beat.

7. Press **Tab** to choose how many bars to keep (1, 2, 4, 8 or 16), then **Ctrl+Enter**. The pattern goes into the bank as a **new song**. The status line says `riff: N bars into song M`. **Ctrl+Z** takes it back.
8. Press **Esc** to close the line and stop the music.

Now play the same code from bm Code.

1. Open a tab of bm Code and type these lines. **[TO FILL: how to open bm Code and a new tab]**

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8" :gain(.9)
bass  = note "<c2 a1 f1 g1>" :s "acid" :lpf(sine:range(300, 1800):slow(4))
```

2. Press **Ctrl+Enter**. In a tab that contains only patterns, the whole tab plays. The status line says `MUSIC 2`: two patterns are playing. The words of the notes that sound light up.
3. Change `kick*4` to `kick*2` and press **Ctrl+Enter** again. The music changes without a pause.
4. Press **Ctrl+.** to stop everything.

If you make a mistake, the line is marked, the status line says what is wrong, and the music that was playing goes on. Fix the line and press **Ctrl+Enter** again.

[TO FILL: shots/riff-sound-line.png]

COPY AND PASTE

```
s "kick*4, ~ snare, hat*8" :gain(.9) :room(.2)
note "c4 e4 g4 c5" :s "epiano" :room(.4)
```

The notation in full

A string is **one cycle** divided into equal parts. This is the mini-notation.

The mini-notation

Written	What it does	Example
`a b c d`	a sequence: four parts of the cycle	`"kick hat snare hat"`
`~` or `-`	a rest	`"kick ~ kick ~"`
`[a b]`	a group: two parts in the space of one	`"kick [hat hat]"`
`a, b`	together (also inside `[ ]`)	`"kick*4, ~ snare"`, `"[c,e,g]"`
`<a b c>`	one per cycle, in turn	`"<c2 a1 f1 g1>"`
`a*3`	three times faster (also `a*<2 4>`)	`"hat*8"`
`a/2`	twice as slow	`"pad/2"`
`a!3`, `a !`	repeated (three times, once more)	`"kick!3 snare"`
`a@3`, `a _ _`	longer: weighs three parts	`"c@3 e"`, `"c _ _ e"`
`a?`, `a?0.3`	plays at random, 50% (or 70%) of the time	`"hat*16?"`
`a(3,8)`, `a(3,8,2)`	Euclidean rhythm: 3 hits spread over 8 steps (rotated by 2)	`"kick(3,8)"`
`{a b c}%4`	polymeter: 4 steps per cycle from the sequence of 3	`"{kick hat snare}%4"`
`a | b`	one of the two at random, every cycle	`"[snare | clap]"`
`a . b c`	groups separated by a dot (`[a] [b c]`)	`"kick . hat hat hat"`
`kick:2`	a variant: two semitones up (for drums)	`"kick:2 kick"`
`kit:2`	a drum of the console's kit (samples): 0 `bd` to 7 `cb`, at its own speed	`"kit:0 kit:2"`, `"bd sd hh"`

Words, notes and numbers

Words are notes (`c`, `eb4`, `f#2`), numbers (`0 2 -1 0.5`) or instrument names.

- Without an octave, a note is in octave 3: `c3` is 48.
- `c4` is 60, middle C, as in Strudel.
- An error in a string says where: `riff: missing ] at 6 in "a [b c"`.

Building patterns

Function	What it does
`s(p)`, `sound(p)`	instruments: a name from the cartridge's bank, a preset (`"kick"`, `"epiano"`, `instruments()`), a number (the bank's sound)
`note(p)`	notes: names or MIDI numbers
`n(p)`	degrees of a scale (`:scale`); without a scale C major from C3
`chord(p)`	chords: `"<C Am F G7>"`, `"Fmaj7"`, `"Dm7"`, `"Bb"`, `"F#m"`, `"Csus4"`, `"Edim"`, in a voicing close to middle C
`seq(a, b, ...)`, `cat(a, b, ...)`, `stack(a, b, ...)`	in sequence within the cycle, one cycle each, together
`timecat({3, a}, {1, b})`	in sequence with weights
`arrange({4, a}, {4, b})`	sections of several cycles, one after the other
`run(n)`	`0 1 2 ... n-1`
`pure(v)`, `silence`, `mini(s)`	one value per cycle, nothing, a mini-notation string
`piece(t)`	a piece from `ai.music` (or from `R.bake`) as a pattern: one cycle every 16 steps

An argument that is a string is always mini-notation, so almost anything can be a pattern: `:lpf("<400 2000>")`, `:fast("<1 2>")`, `:scale("<C:major A:minor>")`.

Changing patterns

Methods chain with the colon (in Lua `p:fast(2)`). With a string the parentheses can be omitted: `note "c e g" :s "pluck"`.

Method	What it does
`:fast(n)`, `:slow(n)`	faster, slower
`:early(t)`, `:late(t)`	earlier, later (in cycles)
`:rev()`, `:palindrome()`	reversed, every other cycle
`:ply(n)`	each event repeated n times
`:iter(n)`, `:iterBack(n)`	each cycle starts one n-th further on
`:every(n, f)`, `:lastOf(n, f)`	f on the first (or last) of every n cycles
`:sometimes(f)`, `:often(f)`, `:rarely(f)`, `:almostNever(f)`, `:almostAlways(f)`, `:sometimesBy(x, f)`, `:someCycles(f)`	f on a part of the events (or cycles) at random
`:degrade()`, `:degradeBy(x)`	randomly removes half (or a part x) of the events

Method	What it does
`:euclid(k, n, [rot])`, `:struct("x ~ x x")`, `:mask("1 0 1 1")`	the structure of a rhythm, or only the events where the mask is true
`:off(t, f)`, `:superimpose(f)`, `:layer(f, g, ...)`	together with a changed copy (shifted by t)
`:jux(f)`, `:juxBy(w, f)`	the pattern on the left, f of the pattern on the right
`:chunk(n, f)`	f on one n-th of the cycle, a different one each time
`:linger(x)`	repeats the first part x of the cycle
`:inside(n, f)`, `:outside(n, f)`	f on a piece of 1/n (or n) cycles
`:swing(n)`, `:swingBy(x, n)`	the even steps of every 1/n a little late
`:segment(n)`	n events per cycle from a signal
`:range(a, b)`, `:rangex(a, b)`	from 0..1 to a..b (exponential)
`:add(x)`, `:sub(x)`, `:mul(x)`, `:div(x)`, `+`, `-`, `*`	on numbers, on the note (or on `n`)
`:transpose(x)`, `:scale("C:minor")`	semitones; the `n` degrees on a scale
`:arp("up")`	the notes of a chord one after the other: `up`, `down`, `updown`, `downup`, `converge`, `diverge`, `thumbup`
`:fmap(f)`	a function on the values

The functions to pass to `every`, `sometimes`, `off` are Lua functions (`function(p) return p:fast(2) end`). There are also shortcuts: `rev`, `fast(2)`, `slow(2)`, `ply(2)`, `late(0.25)`, `add(12)`, `degrade`. So you can write `:every(4, fast(2))` or `:sometimes(add(12))`.

Scales and signals

- **Scales**: `"C:major"`, `"A:minor"`, `"D4:dorian"`, and `phrygian`, `lydian`, `mixolydian`, `locrian`, `harmonic`, `melodic`, `pentatonic`, `minor_pentatonic`, `blues`, `chromatic`, `wholetone`. The root is at octave 3 if you give no octave.
- **Signals** (0..1 over each cycle, continuous; used as values or with `:segment`): `sine`, `cosine`, `saw`, `isaw`, `tri`, `square`, `rand`, `perlin`, `irand(n)`, `choose(a, b, ...)`.

Instruments and effects

Choosing the instrument

`:s(x)` sets the instrument. It can be a name from the cartridge's bank, a preset such as `"kick"` or `"epiano"`, or a number. `instruments()` lists the presets. **[TO FILL: the list of ready-made instrument names, with the output of instruments()]**

- An instrument without a name is `"triangle"`.
- Drums without a note play at their own note (kick 36, snare 50, hat 72); the rest play at C3.

The sound of each note

Control	What it does
`:s(x)`	the instrument (also a pattern: `:s("<pluck epiano>")`)
`:note(x)`, `:n(x)`	note, degree
`:gain(x)`, `:velocity(x)`	the volume (0..1)
`:legato(x)`, `:clip(x)`	how long the note lasts relative to its space (1 = all of it)
`:lpf(hz)`, `:hpf(hz)`, `:bpf(hz)`, `:cutoff(hz)`, `:res(x)`	the filter (low-pass, high-pass, band-pass) and resonance 0..1
`:pan(x)`	-1 left, 1 right
`:room(x)`, `:delay(x)`	how much goes to the room and to the echo (`reverb()`, `echo()` set them)
`:attack(ms)`, `:decay(ms)`, `:sustain(x)`, `:release(ms)`	the envelope
`:shape(x)`, `:drive(x)`, `:noise(x)`	saturation, noise
`:fenv(octaves)`, `:fdecay(ms)`, `:lfo(hz)`, `:wah(octaves)`, `:pwm(x)`, `:duty(x)`	filter envelope, slow oscillation
`:wave(name)`	the waveform (`"saw"`, `"fm"`, `"pluck"`)
`:vib(cent)`, `:vibhz(hz)`, `:detune(cent)`, `:pitch(semitones)`, `:ptime(ms)`	vibrato and pitch envelope
`:fm(x)`, `:fmh(x)`	FM depth and ratio
`:raw(true)`	the 8-bit voice
`:crush(bits)`, `:coarse(n)`	bit crush (1..15 bits kept), the rate divided (1..16)
`:vowel("a")`	the formants of a vowel: `"a"`, `"e"`, `"i"`, `"o"`, `"u"`
`:chorus(x)`, `:trem(x)`, `:duck(x)`	the chorus send, the tremolo (on `:lfo`), the ducking of the other voices (0..1)
`:begin(x)`	where a sample starts (0..1 of its length)

Control	What it does
`:tone{...}`	any key of `tone()` : `:tone{curve = "fold", color = "pink", density = .2, reverse = true}`

BEWARE
The units are those of `tone()` : milliseconds, pan from -1 to 1, resonance 0 to 1. They are not Strudel's, where attack is in seconds and pan goes from 0 to 1.

The kit

`s "kit:2"` (or `s "kit" :n "<0 2>"`) picks a drum of the console's kit, played at its own speed (C4 when no note is given). `pump` , the kit's kick that ducks the rest, works the same way.

- The drums by name, as in Strudel: `s "bd sd hh oh cp cb"` . `sd:2` is two semitones up, as for every drum.
- `rim` and `tom` are the synthesizer's presets. The kit's own are `kit:5` and `kit:6` .
- `R.bake` keeps `kit:0` , `kit:1` as instruments of their own.

COPY AND PASTE
```
beat = s "bd [~ bd] ~ bd, ~ cp, hh*8?" :crush(8)
pad  = chord "<Am F>" :s "choir" :vowel("<a o>") :chorus(.5) :trem(.3)
```

Patterns, songs and structure

Several parts together

In RIFF a song is a set of **named patterns** that play together. Each name is a part: drums, bass, keys.

- With `R.play(name, p)` you give a pattern a name. Playing under an existing name replaces what was there.
- With `R.code`, every global that you give a pattern plays under that name (`drums = s "kick*4"`).
- `stack(a, b, ...)` puts patterns together in one pattern.

Verse and chorus

`arrange({4, a}, {4, b})` puts sections of several cycles one after the other: here four cycles of `a`, then four of `b`.

`<a b c>` changes at every cycle, `cat(a, b, ...)` and `timecat({3, a}, {1, b})` do the same on whole patterns.

COPY AND PASTE

```
local verse  = chord "<Am F C G>" :s "pad"
local bridge = chord "<F C G Am>" :s "pad"
song = arrange({4, verse}, {4, bridge})
```

Repeating and changing

`:every(4, fast(2))` changes one cycle in four. `:sometimes(add(12))` raises some notes by an octave at random. A song that repeats but never sounds the same is made with these.

Tempo

Call	What it does
`setcps(x)`	cycles per second (0.5 at first)
`setcpm(x)`	cycles per minute: `setcpm(30)` is 30 cycles per minute
`bpm(x, [beats])`	beats per minute, four to a cycle: `bpm(120)`

The speed changes without skipping a beat. In `R.code` write them without `R.`; in a game, `R.setcpm(30)`.

Turning a pattern into a song of the bank

In **bm Sound**, **Ctrl+Enter** on the F7 line puts 1, 2, 4, 8 or 16 bars (Tab) in the bank as a new song with its patterns, at the tempo of the riff (16 steps per cycle).

- Instruments the bank lacks become bank sounds.
- One note per step and per track, up to 8 tracks. The filter, position and room of each note stay those of the instrument.

Using a RIFF song in a game

The three calls every game needs

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
  R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid" :lpf(R.sine:range(300, 1800):slow(4)))
end

function _update()
  R.update()                    -- every frame: the notes of the next moments into the queue
end
```

1. `local R = require "riff"` at the top of `main.lua` .
2. `R.play(name, pattern)` once, in `_init` or when the scene changes.
3. `R.update()` in `_update` , **every frame**. Without it nothing plays.

What a slow frame does

A slow frame does not lose the beat. After a pause (the game suspended) the notes already past are skipped.

The functions of the game

Function	What it does
`R.play(name, p)` , `p:play(name)`	plays p under that name (replacing what was there)
`R.stop(name)` , `R.hush()`	stops one, all
`R.update()`	to call in `_update` : queues the notes of the next moments
`R.setcps(x)` , `R.setcpm(x)` , `R.bpm(x, [beats])`	the speed (changes without skipping)
`R.playing()` , `R.get([name])`	the names playing; the pattern of a name (without a name: all together)
`R.now()`	the cycle now
`R.voices(v, ...)`	the voices it can use (none given: all 8)
`R.errors()`	the patterns' errors (a pattern with an error stops)
`R.code(text)`	live code
`R.bake(p, {cycles=, steps=16})`	the pattern as a piece for the bank (the shape of `ai.music`)
`R.piece(t)`	a piece of `ai.music` as a pattern

Live code from the game

`R.code(text)` runs a piece of Lua where RIFF's functions are global (`note` , `s` , `stack` , `sine` , `setcpm` , `hush`) and then the game's.

- **Every global given a pattern plays under that name.**
- A name that starts with `_` does not play: `_drums = ...` silences it.
- The patterns of a previous `R.code` that no longer appear stop.
- A `local` is just a piece.
- It returns `true` , or `nil` and the error. When there is an error, the previous music keeps playing.
- Before playing, it tries each pattern on its first cycle: an error (a note that does not exist, a wrong scale) shows up at once.

Voices, sound effects and bank songs

RIFF uses the synthesizer voices. The voice is chosen when the note starts: a free one, the tail of a sound, or RIFF's oldest note. It never takes the voices of a bank song or of a sound effect. `R.voices(4, 5, 6, 7)` limits the voices it can use.

The low-level call behind it is `play_at(t, sound, [note], [ms], [vol], [tag])` , with the sound's clock `audio_time()` ; `play_cancel([tag])` forgets waiting notes. RIFF uses them for you.

TRY IT

Make a game with the code above and run it. You hear the drums and the bass. Then call `R.hush()` from your code: the music stops. **[TO FILL: a tried example of a game event (a button, a level change) that calls R.stop or R.play]**

Examples

All of these run in the tests (`make test-riff`).

A beat, a bass and keys

COPY AND PASTE

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8" :gain(.9)
bass  = note "<c2 a1 f1 g1>" :s "acid" :lpf(sine:range(300, 1800):slow(4))
keys  = chord "<Cm7 Ab Eb Bb>" :s "epiano" :arp("updown") :fast(2) :room(.4)
```

A pad, an arpeggio and drums

COPY AND PASTE

```
setcpm(28)
local verse  = chord "<Am F C G>"
pad   = verse :s "pad" :room(.6) :gain(.7)
arp   = verse :s "pluck" :arp("updown") :fast(4) :pan(sine:range(-.5, .5))
drums = s "kick ~ ~ kick, ~ snare, hat*8?" :every(4, fast(2))
```

A melody on a scale

COPY AND PASTE

```
setcpm(30)
drums = s "kick*4, ~ snare, hat*8"
bass  = note "<c2 a1 f1 g1>*2" :s "acid" :lpf(800)
lead  = n "0 2 4 <7 6>" :scale("A:minor") :s "pluck" :sometimes(add(12))
```

The kit and the effects

COPY AND PASTE

```
setcpm(30)
drums = s "bd sd [~ bd] sd, hh*8, kit:3(3,8)" :crush(8)
pad   = chord "<Am F>" :s "choir" :vowel("<a o>") :chorus(.5) :trem(.3)
bass  = note "<a1 f1>*4" :s "bitbass" :coarse(4)
```

One-liners for the F7 line

COPY AND PASTE

```
s "kick ~ ~ kick, ~ snare, [~ hat]*4" :swing(4)
note "<c2 c2 eb2 g1>*4" :s "acid" :lpf(sine:range(300, 1800):slow(4))
chord "<Am F C G>" :s "epiano" :arp("updown") :fast(4)
n "0 2 4 <7 6> 4 2 1 ~" :scale("A:minor") :s "pluck"
s "kick(3,8), hat(7,16), ~ clap"
note "c4 e4 g4 c5" :s "chip" :every(4, rev) :fast(2)
```

Stopping and starting from code

COPY AND PASTE

```
R.play("theme", R.note("c4 e4 g4 c5"):s("epiano"):room(0.4))
R.play("beat", R.s("kick ~ kick snare"))
R.hush()
```

Common mistakes

What you see	Why	What to do
Nothing plays in a game	`R.update()` is not called	call it in `_update`, every frame
`riff: missing ] at 6 in "a [b c"`	a bracket is not closed	close it; the message says where
An error with a note word	the word is no note, for example `note "zz"`	use names like `c`, `eb4`, `f#2` or numbers
A pattern stops by itself	an unknown instrument, for example `s "nothing*4"`: the pattern stops when it plays	read `R.errors()`, use a name from `instruments()` or from the bank
A line of `R.code` is silent	the name starts with `_`, or it is a `local`	give it a global name without `_`
The old music goes on after a typo	`R.code` returns `nil` and the error, and keeps the previous music	fix the error and run it again
A part you removed from the code stops	the patterns of the earlier `R.code` that no longer appear stop	write every part you want in each `R.code`
The attack or pan sounds wrong	the units are those of `tone()`: milliseconds, pan -1..1	do not use Strudel's seconds or 0..1 pan
A drum has the wrong pitch	drums without a note use their own note	use `kick:2` to go up two semitones

Open points

- **Your first song in 5 minutes**: how to start bm Sound from the console menu
- **Your first song in 5 minutes**: how to open bm Code and a new tab
- **Your first song in 5 minutes**: picture to take: shots/riff-sound-line.png
- **Instruments and effects**: the list of ready-made instrument names, with the output of instruments()
- **Using a RIFF song in a game**: a tried example of a game event (a button, a level change) that calls R.stop or R.play